



**Министерство науки и высшего образования Российской
Федерации**

**Федеральное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 1

по дисциплине «>

Тема

Студент Прохоров С.Р.

Группа ИУ7-Х6Б

Преподаватель

Москва, 2026

Содержание

ВВЕДЕНИЕ	4
1 Аналитическая часть	5
1.1 Анализ предметной области	5
1.1.1 Сравнительный анализ существующих конкурентов	5
1.2 Формализация задачи	7
1.3 Формализация данных	7
1.4 Формализация ролей	10
1.5 Анализ моделей баз данных	11
2 Конструкторская часть	14
2.1 Описание сущностей базы данных	14
2.2 Ролевая модель	19
2.3 Используемый триггер	19
3 Технологическая часть	21
4 Исследовательская часть	22
4.1 Вывод	22
ЗАКЛЮЧЕНИЕ	23
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	24
Приложение А	25

ВВЕДЕНИЕ

<цель и задачи; можно вводное слово к работе>

1 Аналитическая часть

1.1 Анализ предметной области

Организация и проведение спортивных турниров – это сложный многоуровневый процесс, требующий координации множества участников и ресурсов. Предметная область охватывает планирование соревнований, регистрацию участников, составление расписания матчей, назначение судей и фиксацию результатов.

Традиционно многие организаторы любительских и полупрофессиональных турниров используют для учета электронные таблицы (например, MS Excel) или бумажные носители. Это приводит к ряду проблем:

- Высокая вероятность человеческой ошибки при вводе данных и подсчете очков;
- Сложность оперативного обновления турнирных таблиц и сеток;
- Отсутствие единого информационного пространства для игроков, судей и болельщиков;
- Трудности в оповещении участников об изменениях в расписании.

Внедрение автоматизированной системы управления турнирами (и соответствующей базы данных) позволит централизованно хранить информацию, автоматически создавать турнирные сетки, рассчитывать статистику и обеспечивать мгновенный доступ к результатам.

1.1.1 Сравнительный анализ существующих конкурентов

На российском рынке существует ряд популярных решений для управления спортивными турнирами. Для анализа выбраны три известные платформы: **Наградион**, **Goalstream** и **JoinSport**.

Анализ отечественных конкурентов показывает, что существующие решения либо слишком сложны и ориентированы на официальные федерации (Наградион), либо имеют узкую специализацию и перегружены социальными функциями (Goalstream). Проектируемая база данных и приложение должны взять лучшее из этих систем: гибкость в настройке форматов и четкое разделение ролей, при этом оставаясь интуитивно понятными для организаторов небольших и средних турниров. Основной упор в разрабатываемой БД будет

Таблица 1.1 — Сравнительный анализ систем управления турнирами

Критерий	Наградион	Goalstream	JoinSport
Целевая аудитория	Региональные федерации, крупные лиги.	Любительские командные виды спорта (чаще футбол).	Организаторы турниров, спортивные школы.
Поддержка форматов	Максимально гибкая, для множества видов спорта.	Круговая система, плей-офф, группы.	Групповые этапы, плей-офф, шахматная система.
Пользовательские роли	Администратор, судья, статистик, игрок.	Игрок, менеджер, организатор, болельщик.	Администратор, тренер, игрок, судья.
Интерфейс	Перегруженный интерфейс из-за обилия настроек.	Удобный интерфейс с акцентом на социальные сети.	Современный интерфейс, конструктор сайтов.
Интеграция данных	Выгрузка официальных протоколов, статистика.	Базовый экспорт, есть мобильное приложение.	Хороший API, виджеты для сторонних сайтов.
Недостатки	Высокий порог вхождения, сложность для малых турниров.	Упор на функционал соцсети, узкая специализация.	Платный доступ к модулям, избыточность для разовых турниров.

сделан на оптимизацию структуры хранения данных команд, игроков, матчей и статистики, что обеспечит высокую скорость работы приложения без лишней функциональной перегруженности.

1.2 Формализация задачи

Необходимо спроектировать базу данных для хранения информации о пользователях, турнирах, командах, игроках, матчах, спортивных площадках и статистике. Требуется разработать приложение, предоставляющее интерфейс для просмотра, добавления, изменения и удаления информации, хранящейся в базе данных. Необходимо реализовать три вида ролей – гость, представитель команды и организатор турниров.

1.3 Формализация данных

Основываясь на анализе предметной области, можно выделить следующие категории данных:

- пользователь;
- турнир;
- организатор турнира;
- команда;
- игрок;
- судья;
- тренер;
- матч;
- спортивная площадка.

Сведения о каждой категории данных содержится в таблице 1.2.

Таблица 1.2 — Категории и сведения о данных

Категория	Сведения
Организатор	Название компании, электронная почта, номер телефона
Спортивная площадка	Название локации, город, вместимость
Команда	Название команды, страна, дата основания
Игрок	Привязка к команде, никнейм, настоящее имя, дата рождения
Тренер	Привязка к команде, имя, фамилия, опыт работы (в годах)
Судья	ФИО, категория, опыт работы (в годах)
Турнир	Организатор, название, вид спорта, дата начала, дата окончания, призовой фонд, статус
Заявка на турнир	Турнир, команда, дата регистрации, статус заявки (ожидает, принята, отклонена)
Матч	Турнир, локация, дата и время проведения, стадия турнира, статус матча, команды-участницы и их счет, назначенные судьи и их роли

На рисунке 1.1 приведена ER-диаграмма сущностей в нотации Чена.

img/er.png

Рисунок 1.1 — ER-диаграмма сущностей

Для корректного проведения турнира необходимо разработать систему учета матчей, которая основана на изменении статуса конкретной игры:

- изначально при формировании расписания или турнирной сетки создается матч со статусом «запланирован»;
- по наступлению даты и времени начала матча, статус меняется на «в игре»;
- после внесения финального счета судьей или организатором матч переводится в статус «завершен»;
- в случае непредвиденных обстоятельств организатор может отменить игру, что приводит к изменению статуса на «отменен»;

1.4 Формализация ролей

В системе выделяются 3 типа пользователей:

- гость – неавторизованный пользователь, обладающий возможностями просматривать список турниров, турнирные таблицы, расписание матчей, составы команд и статистику игроков;
- участник (представитель команды) – авторизованный пользователь, обладающий возможностью просматривать общую информацию, подавать и редактировать заявку своей команды на участие в турнире, управлять списком игроков команды;
- организатор (администратор) — авторизованный пользователь, обладающий возможностью просматривать, добавлять и менять данные турниров, команд, игроков, площадок, формировать расписание матчей, вносить результаты игр и управлять ролями пользователей.

На рисунке 1.2 приведена Use-case диаграмма.

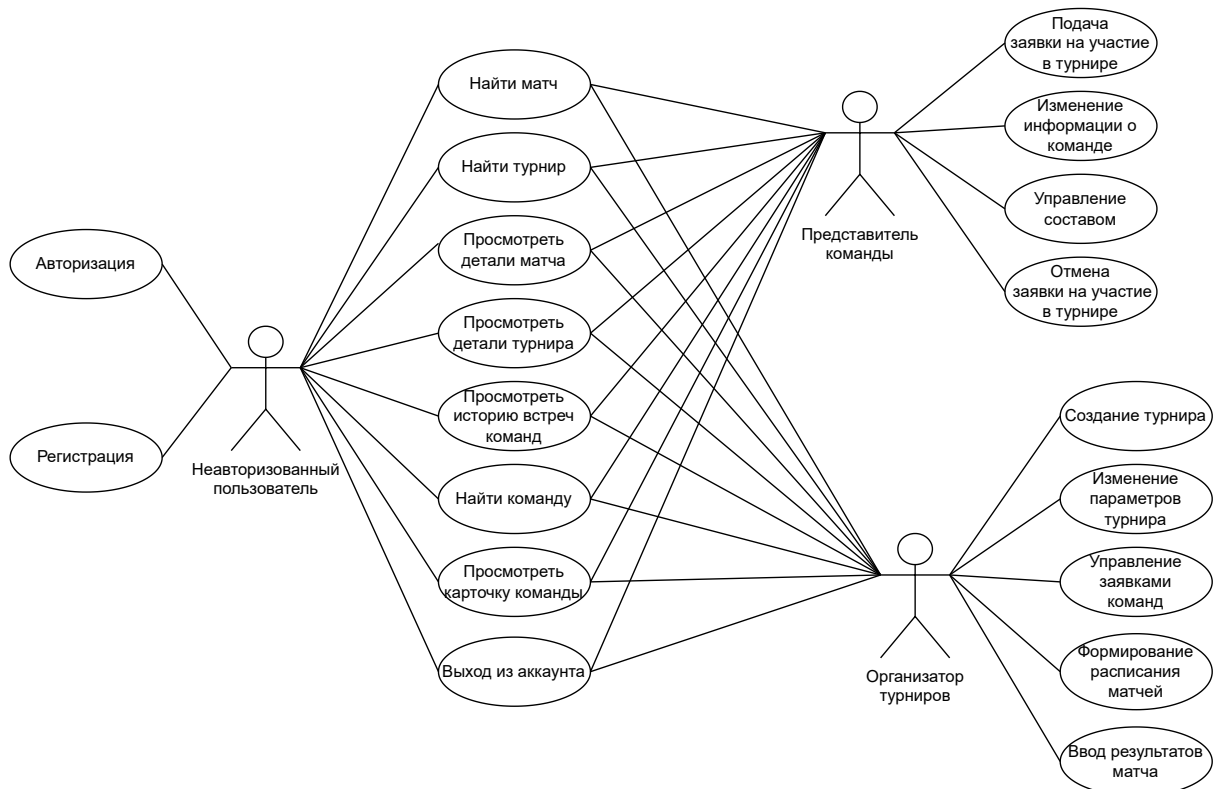


Рисунок 1.2 — Use-case диаграмма

1.5 Анализ моделей баз данных

Основой любой современной информационной системы является база данных – организованная структура для хранения, изменения и извлечения информации. Управление этой структурой осуществляется с помощью систем управления базами данных (СУБД), которые обеспечивают надежность, безопасность и многопользовательский доступ.

При проектировании системы управления спортивными турнирами критически важным этапом является выбор подходящей модели данных, так как от нее зависит производительность приложения, сложность разработки и возможность дальнейшего масштабирования и построения сложной аналитики (например, турнирных таблиц). Рассмотрим основные концептуальные модели организации баз данных.

Иерархическая и сетевая модели

Исторически первыми моделями организации данных были иерархическая и сетевая. В иерархической модели данные представлены в виде строгой древовидной структуры, где каждая запись-потомок имеет строго одного родителя. Сетевая модель расширяет этот подход, позволяя записи иметь нескольких родителей, образуя сложный граф.

Несмотря на высокую скорость навигации по жестко заданным физическим связям, эти подходы обладают существенным недостатком: они крайне негибки при изменении требований. В контексте спортивных турниров присутствуют сложные связи типа «многие-ко-многим» (например, команды участвуют в множестве турниров, а турниры включают множество команд; игроки могут менять клубы между сезонами). Описание таких взаимодействий с помощью иерархий сильно усложняет логику приложения, поэтому сегодня эти модели практически не применяются в новых проектах.

Нереляционные модели (NoSQL)

В последние годы для решения специфических задач активно используются нереляционные модели данных. Наиболее популярным представителем является документоориентированная модель (например, СУБД MongoDB), где

информация хранится в виде гибких JSON-документов.

Документоориентированные БД отлично справляются с неструктурированной информацией и позволяют легко менять схему данных «на лету». Однако для системы управления турнирами этот плюс оборачивается минусом. Отсутствие жесткой схемы повышает риск нарушения целостности данных (например, возможность сохранить матч без указания команд или с несуществующим турниром). Кроме того, документоориентированные базы хуже справляются со сложными транзакциями, затрагивающими множество сущностей одновременно.

Реляционная модель

Реляционная модель, предложенная Э. Коддом, является стандартом для большинства современных учетных систем. Данные в ней представляются в виде двумерных таблиц (отношений), состоящих из строк (кортежей) и столбцов (атрибутов). Логические связи между таблицами устанавливаются с помощью первичных (Primary Key) и внешних (Foreign Key) ключей.

Ключевые преимущества реляционной модели для разрабатываемой системы:

- **Строгая структурированность и целостность:** механизм внешних ключей не позволит удалить команду, если у нее есть сыгранные матчи в системе, что защищает историю турнира от разрушения.
- **Поддержка ACID-транзакций:** гарантирует, что процессы, состоящие из нескольких шагов (например, фиксация результатов матча, начисление очков командам и запись статистики игроков), выполняются атомарно – либо полностью, либо никак.
- **Мощный аппарат агрегации (SQL):** реляционная алгебра идеально подходит для динамического расчета турнирного положения. С помощью SQL-запросов можно легко группировать данные, считать сумму забитых голов, разницу мячей и набранные очки.

Вывод по разделу

В ходе анализа предметной области проведения спортивных соревнований были определены требования к разрабатываемой системе, выделены ключевые

информационные сущности (турниры, команды, игроки, матчи) и формализованы роли пользователей.

Анализ специфики данных показал, что информация в спортивных турнирах обладает высокой степенью связности и строгой математической логикой. Матч не может существовать вне турнира, статистика привязана к конкретным игрокам, а результаты игр напрямую влияют на турнирное положение команд. Основываясь на необходимости обеспечения строгой ссылочной целостности, поддержки сложных аналитических агрегаций и транзакционной надежности, в качестве оптимальной модели хранения данных для разрабатываемой системы была выбрана **реляционная модель**.

Вывод

<что сделали в анализе, кратко>

2 Конструкторская часть

2.1 Описание сущностей базы данных

В соответствии с таблицей 1.2, содержащей данные, которые должны находиться в базе данных, можно выделить следующие таблицы:

- 1) organizer – таблица организаторов турниров;
- 2) location – таблица площадок (локаций) проведения матчей;
- 3) team – таблица спортивных команд;
- 4) referee – таблица судей;
- 5) tournament – таблица спортивных турниров;
- 6) match – таблица матчей турнира;
- 7) player – таблица игроков команд;
- 8) coach – таблица тренеров команд;
- 9) tournament_registration – таблица регистраций команд на турнир;
- 10) match_participant – таблица команд-участников матчей и их результатов;
- 11) match_referee – таблица назначения судей на конкретные матчи.

На основе информации о выбранной СУБД и представленной на рисунке 1.1 диаграмме сущность-связь, мы определим для каждой из указанных таблиц структуру столбцов, их типы и установим соответствующие ограничения, которые представлены в таблицах 2.1–2.11.

Таблица 2.1 — Информация о столбцах таблицы организаторов (organizer)

Столбец	Тип данных	Ограничения	Значение
id	bigint	NOT NULL, PRIMARY KEY	Идентификатор организатора
company_name	text	NOT NULL	Название компании
email	text	NOT NULL, UNIQUE, CHECK (формат email)	Электронная почта
phone	text	—	Номер телефона

Таблица 2.2 — Информация о столбцах таблицы площадок (location)

Столбец	Тип данных	Ограничения	Значение
id	bigint	NOT NULL, PRIMARY KEY	Идентификатор площадки
name	text	NOT NULL	Название
city	text	NOT NULL	Город
capacity	bigint	CHECK (capacity > 0)	Вместимость

Таблица 2.3 — Информация о столбцах таблицы команд (team)

Столбец	Тип данных	Ограничения	Значение
id	bigint	NOT NULL, PRIMARY KEY	Идентификатор команды
name	text	NOT NULL	Название команды
country	text	NOT NULL	Страна
foundation_date	date	—	Дата основания

Таблица 2.4 — Информация о столбцах таблицы судей (referee)

Столбец	Тип данных	Ограничения	Значение
id	bigint	NOT NULL, PRIMARY KEY	Идентификатор судьи
full_name	text	NOT NULL	ФИО судьи
category	text	NOT NULL	Категория
experience_years	int	DEFAULT 0, CHECK (≥ 0)	Опыт работы (в годах)

Таблица 2.5 — Информация о столбцах таблицы турниров (tournament)

Столбец	Тип данных	Ограничения	Значение
id	bigint	NOT NULL, PRIMARY KEY	Идентификатор турнира
organizer_id	bigint	NOT NULL, FK (organizer) ON DELETE CASCADE	Идентификатор организатора
name	text	NOT NULL	Название турнира
sport_type	text	NOT NULL	Вид спорта
start_date	date	NOT NULL, CHECK ($\leq$ end_date)	Дата начала
end_date	date	NOT NULL	Дата окончания
prize_pool	numeric	DEFAULT 0.00, CHECK ($\geq$ 0)	Призовой фонд
status	text	DEFAULT 'registration', CHECK (status IN ...)	Статус турнира

Таблица 2.6 — Информация о столбцах таблицы матчей (match)

Столбец	Тип данных	Ограничения	Значение
id	bigint	NOT NULL, PRIMARY KEY	Идентификатор матча
tournament_id	bigint	NOT NULL, FK (tournament) ON DELETE CASCADE	Идентификатор турнира
location_id	bigint	FK (location) ON DELETE SET NULL	Идентификатор площадки
match_date	timestamp	NOT NULL	Дата и время проведения
stage_name	text	NOT NULL	Стадия турнира
status	text	DEFAULT 'scheduled', CHECK (status IN ...)	Статус матча

Таблица 2.7 — Информация о столбцах таблицы игроков (player)

Столбец	Тип данных	Ограничения	Значение
id	bigint	NOT NULL, PRIMARY KEY	Идентификатор игрока
team_id	bigint	FK (team) ON DELETE SET NULL	Идентификатор команды
nickname	text	NOT NULL	Никнейм
real_name	text	NOT NULL	Настоящее имя
birth_date	date	NOT NULL	Дата рождения

Таблица 2.8 — Информация о столбцах таблицы тренеров (coach)

Столбец	Тип данных	Ограничения	Значение
id	bigint	NOT NULL, PRIMARY KEY	Идентификатор тренера
team_id	bigint	FK (team) ON DELETE SET NULL	Идентификатор команды
first_name	text	NOT NULL	Имя
last_name	text	NOT NULL	Фамилия
experience_years	int	DEFAULT 0, CHECK (≥ 0)	Опыт работы (в годах)

Таблица 2.9 — Информация о столбцах таблицы регистрации на турнир (tournament_registration)

Столбец	Тип данных	Ограничения	Значение
tournament_id	bigint	NOT NULL, PK, FK (tournament) ON DELETE CASCADE	Идентификатор турнира
team_id	bigint	NOT NULL, PK, FK (team) ON DELETE CASCADE	Идентификатор команды
registration_date	timestamp	DEFAULT current_timestamp	Дата регистрации
status_registration	text	DEFAULT 'pending', CHECK (status IN ...)	Статус заявки

Таблица 2.10 — Информация о столбцах таблицы участников матча (match_participant)

Столбец	Тип данных	Ограничения	Значение
match_id	bigint	NOT NULL, PK, FK (match) ON DELETE CASCADE	Идентификатор матча
team_id	bigint	NOT NULL, PK, FK (team) ON DELETE CASCADE	Идентификатор команды
score	int	DEFAULT 0, CHECK (≥ 0)	Счет команды

Таблица 2.11 — Информация о столбцах таблицы судей матча (match_referee)

Столбец	Тип данных	Ограничения	Значение
match_id	bigint	NOT NULL, PK, FK (match) ON DELETE CASCADE	Идентификатор матча
referee_id	bigint	NOT NULL, PK, FK (referee) ON DELETE CASCADE	Идентификатор судьи
role	text	NOT NULL	Роль судьи

2.2 Ролевая модель

Для реализации описанной логики на уровне базы данных (PostgreSQL) проектируется ролевая модель, включающая 3 роли базы данных. Разграничение доступа к объектам (таблицам и представлениям) выглядит следующим образом:

- 1) Неавторизованный пользователь – имеет права доступа только на чтение к таблицам tournament, match, team, player, location, match_participant. Запрещены любые операции изменения данных.
- 2) Представитель команды – те же права, что и у неавторизованного пользователя, при этом добавляются возможности для добавления, обновления и удаления данных из таблиц tournament_registration player
- 3) Организатор турнира – имеет полный доступ в базе данных.

2.3 Используемый триггер

В базе данных будет присутствовать триггер, который активируется автоматически после завершения матча-финала турнира. Этот триггер автоматически завершает турнир, чей финал был окончен, что позволяет обеспечить актуальность данных.

Вывод

<что сделали в конструкторке и получили в результате, кратко>

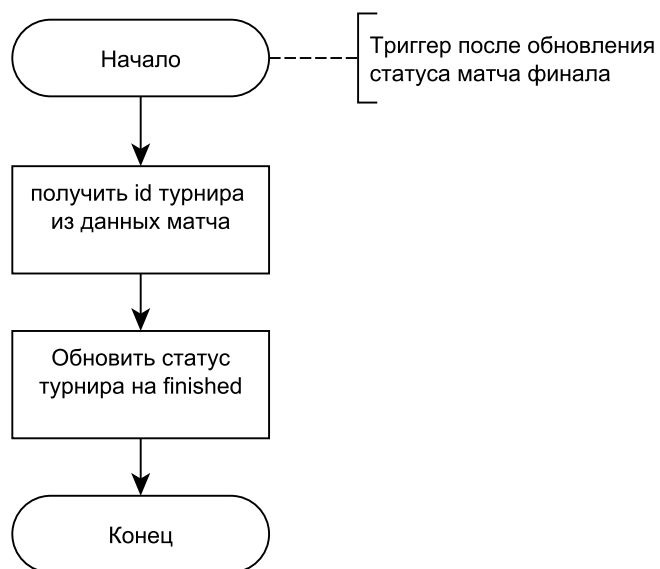


Рисунок 2.1 — Схема алгоритма триггера, обновляющего статус турнира после конца матча-финала

3 Технологическая часть

<писать все про реализацию: какие языки, какие ide и тд.; коды алгоритмов/программы> Для реализации данной лабораторной работы использовался язык Python 3.12.3, в редакторе VScode.

<тестовые данные и волшебные слова «все тесты пройдены успешно» >

Вывод

<что делали и получили в результате>

4 Исследовательская часть

<цель исследования, на какой машине делали (указать ЦПУ, ОЗУ, ОС), желательно написать, как исследовали и при каких условиях; что получили в результате (таблицы + графики)>

Вычисления производились на Windows 11, ОЗУ – 16Гб, процессор – AMD Ryzen 5 6600H.

4.1 Вывод

<что сделали и получили в результате, кратко>

ЗАКЛЮЧЕНИЕ

<копируем цель и кратко описываем, что делали и получили в результате;
указать, какие задачи выполнили>

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Литература 1.
2. Литература 2.
3. ...
4. Литература N.

Приложение А

<тут всякие слишком большие ништяки, которые вы хотели бы добавить
в отчет>